

Question Bank — Unit 2 | Complete Answer Guide

Topics Covered:

- ◆ DDL Commands: CREATE, ALTER, DROP
- ◆ Constraints: Primary Key, Foreign Key, CHECK
- ◆ SQL Queries: SELECT, INSERT, UPDATE, DELETE
- ◆ Clauses: WHERE, GROUP BY, HAVING, EXISTS, IN, LIKE
- ◆ Aggregate Functions, Views, Triggers, Stored Procedures
 - ◆ Normal Forms: 1NF, 2NF, 3NF
- ◆ Database Design Guidelines & Anomalies
- ◆ Functional Dependencies, Inference Rules, Minimal Cover
 - ◆ Lossless Join & Equivalence of FD Sets

Q1. DDL (Data Definition Language) Statements — CREATE TABLE, ALTER TABLE

DDL (Data Definition Language) is used to define and modify the structure of database objects such as tables, schemas, indexes, and views. DDL statements are auto-committed, meaning changes take effect immediately.

1.1 CREATE TABLE

Creates a new table with specified columns and constraints.

Syntax:

```
CREATE TABLE table_name (  
    column1 datatype [constraint],  
    column2 datatype [constraint],  
    ...  
    [table_constraints]  
);
```

Example — Student Table:

```
CREATE TABLE Student (  
    student_id INT NOT NULL,  
    name VARCHAR(50) NOT NULL,  
    dob DATE,  
    branch VARCHAR(30),  
    percentage DECIMAL(5,2),  
    PRIMARY KEY (student_id)  
);
```

Example — Employee Table:

```
CREATE TABLE Employee (  
    emp_id INT NOT NULL,  
    emp_name VARCHAR(100) NOT NULL,  
    department VARCHAR(50),  
    salary DECIMAL(10,2),  
    manager_id INT,  
    PRIMARY KEY (emp_id),  
    FOREIGN KEY (manager_id) REFERENCES Employee(emp_id)  
);
```

1.2 ALTER TABLE

Modifies the structure of an existing table.

```
-- Add a new column  
ALTER TABLE Student ADD COLUMN email VARCHAR(100);  
  
-- Modify a column datatype  
ALTER TABLE Student MODIFY COLUMN percentage FLOAT;  
  
-- Drop a column  
ALTER TABLE Student DROP COLUMN dob;
```

```
-- Rename a column
ALTER TABLE Student RENAME COLUMN name TO student_name;

-- Add a constraint
ALTER TABLE Employee ADD CONSTRAINT chk_sal CHECK (salary > 0);

-- Drop a constraint
ALTER TABLE Employee DROP CONSTRAINT chk_sal;
```

1.3 DROP TABLE

```
DROP TABLE Student; -- permanently removes the table
DROP TABLE IF EXISTS Temp; -- safe drop (no error if not exists)
```

1.4 TRUNCATE TABLE

```
TRUNCATE TABLE Student; -- removes all rows, keeps structure
```

Q2. Specifying PRIMARY KEY and FOREIGN KEY Constraints

2.1 PRIMARY KEY Constraint

A primary key uniquely identifies each record in a table. It cannot be NULL and must be unique. A table can have only ONE primary key (which may consist of multiple columns — composite key).

```
-- Column-level primary key
CREATE TABLE Customer (
  cust_id INT NOT NULL PRIMARY KEY,
  cust_name VARCHAR(80) NOT NULL
);

-- Table-level (composite) primary key
CREATE TABLE Enrollment (
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  grade CHAR(2),
  PRIMARY KEY (student_id, course_id)
);
```

2.2 FOREIGN KEY Constraint

A foreign key establishes a referential integrity link between two tables. The foreign key column in the child table must match values in the primary key column of the parent table (or be NULL).

```
CREATE TABLE Orders (
  order_id INT NOT NULL PRIMARY KEY,
  cust_id INT,
  amount DECIMAL(10,2),
  FOREIGN KEY (cust_id) REFERENCES Customer(cust_id)
  ON DELETE CASCADE
  ON UPDATE SET NULL
);
```

Referential Actions:

Action	ON DELETE	ON UPDATE
--------	-----------	-----------

CASCADE	Child rows deleted automatically	FK value updated automatically
SET NULL	FK set to NULL	FK set to NULL
SET DEFAULT	FK set to default value	FK set to default value
RESTRICT	Prevents deletion of parent row	Prevents update of parent key
NO ACTION	Similar to RESTRICT (checked at end)	Similar to RESTRICT

Q3. CHECK Constraints

A CHECK constraint limits the range of values that can be inserted into a column. It defines a condition that every row must satisfy. If the condition evaluates to FALSE, the operation is rejected.

Syntax:

```
-- Column-level CHECK
column_name datatype CHECK (condition)

-- Table-level CHECK (can reference multiple columns)
CONSTRAINT constraint_name CHECK (condition)
```

Examples:

```
CREATE TABLE Employee (
  emp_id INT NOT NULL PRIMARY KEY,
  emp_name VARCHAR(80),
  salary DECIMAL(10,2) CHECK (salary >= 10000),
  age INT CHECK (age >= 18 AND age <= 65),
  gender CHAR(1) CHECK (gender IN ('M','F','O')),
  -- Table-level check
  CONSTRAINT chk_sal_age CHECK (salary > 20000 OR age > 30)
);

CREATE TABLE Student (
  student_id INT PRIMARY KEY,
  percentage DECIMAL(5,2) CHECK (percentage >= 0 AND percentage <= 100),
  grade CHAR(2) CHECK (grade IN ('A+', 'A', 'B', 'C', 'D', 'F'))
);
```

Note: CHECK constraints are evaluated only for INSERT and UPDATE operations, not DELETE.

Q4. All SQL Statements — Student, Customer, Employee & Hospital Databases

4.1 Student Database

```
-- Schema
CREATE TABLE Student (student_id INT PRIMARY KEY, name VARCHAR(50),
branch VARCHAR(30), semester INT, percentage DECIMAL(5,2));
CREATE TABLE Course (course_id INT PRIMARY KEY, course_name VARCHAR(80),
credits INT);
CREATE TABLE Enroll (student_id INT, course_id INT,
grade CHAR(2), PRIMARY KEY(student_id, course_id),
FOREIGN KEY(student_id) REFERENCES Student(student_id),
FOREIGN KEY(course_id) REFERENCES Course(course_id));

-- INSERT
INSERT INTO Student VALUES (1, 'Alice', 'CSE', 5, 88.5);
INSERT INTO Student VALUES (2, 'Bob', 'ECE', 3, 72.0);

-- SELECT
SELECT * FROM Student;
SELECT name, percentage FROM Student WHERE branch='CSE';

-- UPDATE
UPDATE Student SET percentage=90.0 WHERE student_id=1;

-- DELETE
DELETE FROM Student WHERE student_id=2;
```

4.2 Customer Database

```
CREATE TABLE Customer (cust_id INT PRIMARY KEY, cust_name VARCHAR(80),
city VARCHAR(40), phone VARCHAR(15));
CREATE TABLE Product (prod_id INT PRIMARY KEY, prod_name VARCHAR(80),
price DECIMAL(10,2), stock INT);
CREATE TABLE Orders (order_id INT PRIMARY KEY, cust_id INT, prod_id INT,
quantity INT, order_date DATE,
FOREIGN KEY(cust_id) REFERENCES Customer(cust_id),
FOREIGN KEY(prod_id) REFERENCES Product(prod_id));

INSERT INTO Customer VALUES (101, 'Alice', 'Mumbai', '9876543210');
INSERT INTO Product VALUES (1, 'Laptop', 55000.00, 20);
INSERT INTO Orders VALUES (1001, 101, 1, 2, '2025-01-15');

SELECT c.cust_name, p.prod_name, o.quantity
FROM Customer c, Product p, Orders o
WHERE c.cust_id=o.cust_id AND p.prod_id=o.prod_id;
```

4.3 Employee Database

```
CREATE TABLE Department (dept_id INT PRIMARY KEY, dept_name VARCHAR(50));
CREATE TABLE Employee (emp_id INT PRIMARY KEY, emp_name VARCHAR(80),
salary DECIMAL(10,2), dept_id INT, manager_id INT,
```

```

FOREIGN KEY(dept_id) REFERENCES Department(dept_id),
FOREIGN KEY(manager_id) REFERENCES Employee(emp_id));

INSERT INTO Department VALUES (10, 'HR'), (20, 'IT'), (30, 'Finance');
INSERT INTO Employee VALUES (1, 'Alice', 80000, 20, NULL);
INSERT INTO Employee VALUES (2, 'Bob', 50000, 20, 1);

SELECT e.emp_name, d.dept_name, e.salary
FROM Employee e JOIN Department d ON e.dept_id=d.dept_id;

```

4.4 Hospital Database

```

CREATE TABLE Doctor (doc_id INT PRIMARY KEY, doc_name VARCHAR(80),
specialization VARCHAR(50));
CREATE TABLE Patient (pat_id INT PRIMARY KEY, pat_name VARCHAR(80),
dob DATE, contact VARCHAR(15));
CREATE TABLE Appointment (appt_id INT PRIMARY KEY, pat_id INT, doc_id INT,
appt_date DATE, diagnosis VARCHAR(200),
FOREIGN KEY(pat_id) REFERENCES Patient(pat_id),
FOREIGN KEY(doc_id) REFERENCES Doctor(doc_id));

INSERT INTO Doctor VALUES (1, 'Dr. Smith', 'Cardiology');
INSERT INTO Patient VALUES (101, 'John', '1990-05-12', '9876543210');
INSERT INTO Appointment VALUES (1001, 101, 1, '2025-03-01', 'Hypertension');

```

Q5. SQL Clauses — EXISTS, IN, LIKE, OR & Nested Queries

5.1 IN Clause

Checks if a value exists in a list or subquery result.

```

-- Students from specific branches
SELECT name FROM Student WHERE branch IN ('CSE', 'IT', 'ISE');

-- Students enrolled in courses 101 or 102
SELECT DISTINCT student_id FROM Enroll
WHERE course_id IN (101, 102);

```

5.2 LIKE Clause

Pattern matching: % = any sequence of chars, _ = single char.

```

SELECT name FROM Student WHERE name LIKE 'A%'; -- starts with A
SELECT name FROM Employee WHERE emp_name LIKE '%son'; -- ends with 'son'
SELECT * FROM Customer WHERE phone LIKE '98_____'; -- starts with 98
SELECT * FROM Patient WHERE pat_name LIKE '_o%'; -- 2nd char is 'o'

```

5.3 OR Clause

```

SELECT * FROM Student WHERE branch='CSE' OR percentage > 85;
SELECT * FROM Employee WHERE salary > 70000 OR dept_id = 10;
SELECT * FROM Patient WHERE doc_id=1 OR doc_id=2;

```

5.4 EXISTS Clause

Returns TRUE if the subquery returns at least one row.

```
-- Students who have enrolled in at least one course
SELECT name FROM Student s
WHERE EXISTS (SELECT 1 FROM Enroll e WHERE e.student_id = s.student_id);

-- Doctors who have at least one appointment
SELECT doc_name FROM Doctor d
WHERE EXISTS (SELECT 1 FROM Appointment a WHERE a.doc_id = d.doc_id);
```

5.5 Nested (Subquery) Examples

```
-- Students with above-average percentage
SELECT name, percentage FROM Student
WHERE percentage > (SELECT AVG(percentage) FROM Student);

-- Employees earning more than their department's average
SELECT emp_name, salary FROM Employee e
WHERE salary > (SELECT AVG(salary) FROM Employee
WHERE dept_id = e.dept_id);

-- Customers who have placed orders (using IN)
SELECT cust_name FROM Customer
WHERE cust_id IN (SELECT cust_id FROM Orders);

-- NOT IN: students not enrolled in any course
SELECT name FROM Student
WHERE student_id NOT IN (SELECT student_id FROM Enroll);
```

Q6. ALTER TABLE, DELETE and UPDATE Commands

6.1 ALTER TABLE

```
-- 1) Add a column
ALTER TABLE Student ADD COLUMN email VARCHAR(100);

-- 2) Modify column type
ALTER TABLE Employee MODIFY COLUMN salary FLOAT;

-- 3) Drop a column
ALTER TABLE Customer DROP COLUMN phone;

-- 4) Rename a column
ALTER TABLE Patient RENAME COLUMN pat_name TO patient_name;

-- 5) Add primary key
ALTER TABLE Student ADD PRIMARY KEY (student_id);

-- 6) Add foreign key
ALTER TABLE Employee ADD FOREIGN KEY (dept_id) REFERENCES Department(dept_id);

-- 7) Add CHECK constraint
ALTER TABLE Employee ADD CONSTRAINT chk_salary CHECK (salary > 0);

-- 8) Rename table
ALTER TABLE Enroll RENAME TO Enrollment;
```

6.2 DELETE Command

Removes rows from a table. Without WHERE, ALL rows are deleted.

```
-- Delete specific row
DELETE FROM Student WHERE student_id = 5;

-- Delete based on condition
DELETE FROM Employee WHERE salary < 15000;

-- Delete with subquery
DELETE FROM Orders WHERE cust_id IN
(SELECT cust_id FROM Customer WHERE city = 'Mumbai');

-- Delete all rows (keeps table structure)
DELETE FROM Appointment;
```

6.3 UPDATE Command

Modifies existing rows. Always use WHERE to avoid updating all rows.

```
-- Update single column
UPDATE Student SET percentage = 92.5 WHERE student_id = 1;

-- Update multiple columns
UPDATE Employee SET salary = 75000, dept_id = 20 WHERE emp_id = 3;

-- Update with expression
UPDATE Employee SET salary = salary * 1.10 WHERE dept_id = 20;

-- Update with subquery
UPDATE Employee SET salary = salary * 1.15
```

```
WHERE dept_id = (SELECT dept_id FROM Department WHERE dept_name='IT');
```

Q7. Aggregate Functions — GROUP BY and HAVING Clause

Aggregate functions perform calculations on a set of values and return a single value. They are commonly used with GROUP BY to group results by one or more columns.

Function	Description	Example
COUNT(*)	Count all rows	COUNT(*) — count of students
COUNT(col)	Count non-NULL values	COUNT(salary)
SUM(col)	Sum of all values	SUM(salary)
AVG(col)	Average of values	AVG(percentage)
MAX(col)	Maximum value	MAX(salary)
MIN(col)	Minimum value	MIN(salary)

```
-- Basic aggregate functions
SELECT COUNT(*), AVG(percentage), MAX(percentage), MIN(percentage)
FROM Student;

-- GROUP BY: average salary per department
SELECT dept_id, AVG(salary) AS avg_salary, COUNT(*) AS emp_count
FROM Employee
GROUP BY dept_id;

-- GROUP BY with JOIN
SELECT d.dept_name, COUNT(e.emp_id) AS total_emp, SUM(e.salary) AS total_sal
FROM Department d JOIN Employee e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;

-- HAVING: departments with average salary > 50000
SELECT dept_id, AVG(salary) AS avg_sal
FROM Employee
GROUP BY dept_id
HAVING AVG(salary) > 50000;

-- Students: branches with more than 5 students, avg % > 70
SELECT branch, COUNT(*) AS students, AVG(percentage) AS avg_pct
FROM Student
GROUP BY branch
HAVING COUNT(*) > 5 AND AVG(percentage) > 70
ORDER BY avg_pct DESC;
```

Note: WHERE filters rows BEFORE grouping; HAVING filters GROUPS AFTER grouping.

Q8. Trigger Commands — BEFORE INSERT and AFTER INSERT

A trigger is a stored program that is automatically executed when a specified event (INSERT, UPDATE, DELETE) occurs on a table. Triggers enforce business rules and maintain data integrity automatically.

General Syntax:

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON table_name
FOR EACH ROW
BEGIN
-- trigger body (SQL statements)
-- NEW.column_name = new value being inserted/updated
-- OLD.column_name = old value before update/delete
END;
```

8.1 BEFORE INSERT Trigger

Fired BEFORE the row is inserted. Used to validate or modify data before it enters the table.

```
-- Automatically set grade based on percentage before insert
DELIMITER $$
CREATE TRIGGER trg_before_insert_student
BEFORE INSERT ON Student
FOR EACH ROW
BEGIN
IF NEW.percentage >= 90 THEN
SET NEW.grade = 'A+';
ELSEIF NEW.percentage >= 75 THEN
SET NEW.grade = 'A';
ELSEIF NEW.percentage >= 60 THEN
SET NEW.grade = 'B';
ELSE
SET NEW.grade = 'C';
END IF;
END$$
DELIMITER ;

-- Prevent inserting negative salary
CREATE TRIGGER trg_check_salary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
IF NEW.salary < 0 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Salary cannot be negative';
END IF;
END;
```

8.2 AFTER INSERT Trigger

Fired AFTER the row is successfully inserted. Used for logging, updating related tables, or sending notifications.

```
-- Log every new patient appointment
CREATE TABLE Appointment_Log (
log_id INT AUTO_INCREMENT PRIMARY KEY,
appt_id INT, log_time DATETIME, action VARCHAR(50));

CREATE TRIGGER trg_after_insert_appt
AFTER INSERT ON Appointment
FOR EACH ROW
BEGIN
INSERT INTO Appointment_Log (appt_id, log_time, action)
VALUES (NEW.appt_id, NOW(), 'INSERTED');
END;

-- After inserting an order, reduce product stock
CREATE TRIGGER trg_after_order_insert
AFTER INSERT ON Orders
FOR EACH ROW
BEGIN
UPDATE Product
SET stock = stock - NEW.quantity
WHERE prod_id = NEW.prod_id;
END;
```

Q9. Creation of Views

A VIEW is a virtual table based on the result of a SQL SELECT query. It does not store data physically; it only stores the query definition. Views simplify complex queries, provide security, and ensure data abstraction.

Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Examples:

```
-- View 1: CSE Students only
CREATE VIEW CSE_Students AS
SELECT student_id, name, percentage
FROM Student
WHERE branch = 'CSE';

-- Query the view like a table
SELECT * FROM CSE_Students WHERE percentage > 80;

-- View 2: Employee-Department summary
CREATE VIEW Emp_Details AS
```

```

SELECT e.emp_id, e.emp_name, d.dept_name, e.salary
FROM Employee e JOIN Department d ON e.dept_id = d.dept_id;

-- View 3: High-salary employees (with CHECK OPTION)
CREATE VIEW High_Earners AS
SELECT * FROM Employee WHERE salary > 60000
WITH CHECK OPTION;

-- Update a view
UPDATE CSE_Students SET percentage = 91.0 WHERE student_id = 1;

-- Drop a view
DROP VIEW CSE_Students;

```

Advantages of Views:

- Simplifies complex queries by hiding complexity
- Provides security by restricting access to specific columns/rows
- Data independence — underlying table structure can change without affecting the view
- Reusability — define once, use many times

Q10. Stored Procedures — Examples

A stored procedure is a precompiled set of SQL statements stored in the database and executed as a unit. They improve performance (compiled once), promote code reuse, and enhance security.

```

-- Basic syntax
DELIMITER $$
CREATE PROCEDURE procedure_name (IN param1 datatype, OUT param2 datatype)
BEGIN
-- SQL statements
END$$
DELIMITER ;

-- Call: CALL procedure_name(arguments);

```

Example 1: Get Student by Branch

```

DELIMITER $$
CREATE PROCEDURE GetStudentsByBranch(IN branch_name VARCHAR(30))
BEGIN
SELECT student_id, name, percentage
FROM Student
WHERE branch = branch_name
ORDER BY percentage DESC;
END$$
DELIMITER ;

-- Usage:
CALL GetStudentsByBranch('CSE');

```

Example 2: Give Salary Raise

```

DELIMITER $$
CREATE PROCEDURE GiveSalaryRaise(IN dept INT, IN pct DECIMAL(5,2))

```

```
BEGIN
UPDATE Employee
SET salary = salary * (1 + pct/100)
WHERE dept_id = dept;
SELECT ROW_COUNT() AS rows_updated;
END$$
DELIMITER ;

CALL GiveSalaryRaise(20, 10); -- 10% raise to dept 20
```

Example 3: OUT parameter

```
DELIMITER $$
CREATE PROCEDURE GetAvgSalary(IN d_id INT, OUT avg_sal DECIMAL(10,2))
BEGIN
SELECT AVG(salary) INTO avg_sal
FROM Employee WHERE dept_id = d_id;
END$$
DELIMITER ;

CALL GetAvgSalary(20, @result);
SELECT @result AS avg_salary;
```

Q11. Explanation of 1NF, 2NF and 3NF with Proper Examples

Normalization is the process of organizing a relational database to minimize redundancy and dependency anomalies. E.F. Codd proposed Normal Forms as standards for schema quality.

11.1 First Normal Form (1NF)

A relation is in 1NF if:

- Every attribute contains only ATOMIC (indivisible) values
- No repeating groups or multivalued attributes
- Each column has a unique name and each cell has a single value

NOT in 1NF (multivalued attribute):

StudentID	Name	Courses
101	Alice	DBMS, OS, CN
102	Bob	DBMS, Math

After conversion to 1NF:

StudentID	Name	Course
101	Alice	DBMS
101	Alice	OS
101	Alice	CN
102	Bob	DBMS
102	Bob	Math

11.2 Second Normal Form (2NF)

A relation is in 2NF if: (1) It is in 1NF, AND (2) Every non-prime attribute is FULLY functionally dependent on the entire primary key (no partial dependencies).

Example — NOT in 2NF:

Relation: Order_Details(OrderID, ProductID, ProductName, Qty)

PK = {OrderID, ProductID}. FDs: {OrderID, ProductID} → Qty BUT ProductID → ProductName (partial!)

Decompose into 2NF:

```
Order_Items(OrderID, ProductID, Qty) -- PK: {OrderID, ProductID}
```

```
Product(ProductID, ProductName) -- PK: ProductID
```

11.3 Third Normal Form (3NF)

A relation is in 3NF if: (1) It is in 2NF, AND (2) No non-prime attribute is transitively dependent on the primary key.

Example — NOT in 3NF:

Relation: Employee(EmpID, EmpName, DeptID, DeptName)

PK = EmpID. FDs: EmpID → DeptID and DeptID → DeptName. So EmpID → DeptName is TRANSITIVE through DeptID.

Decompose into 3NF:

```
Employee(EmpID, EmpName, DeptID) -- PK: EmpID
```

Department(DeptID, DeptName) -- PK: DeptID

Q12. Guidelines of Database Design

Good relational database design follows four informal guidelines to ensure clarity, efficiency, and integrity.

Guideline 1: Clear Semantics of Attributes

Design each relation so its meaning (semantics) is easy to explain. Each relation should represent ONE entity type or ONE relationship type. Do not mix attributes from multiple entity/relationship types.

Note: Example: Bad: EMP_DEPT combining Employee + Department attributes. Good: Separate EMPLOYEE and DEPARTMENT tables.

Guideline 2: Avoid Redundancy & Update Anomalies

Do not group attributes that cause INSERT, DELETE, or MODIFICATION anomalies. Storing the same data in multiple rows wastes space and risks inconsistency.

Note: Example: If department info is repeated in every employee row, changing the manager requires updating ALL rows.

Guideline 3: Minimize NULL Values

Avoid putting attributes that frequently have NULL values into a relation. NULLs waste storage, complicate JOIN operations, and create problems for aggregate functions.

Note: Example: If most employees don't have a 'bonus', don't include 'bonus' in the main Employee table.

Guideline 4: Avoid Spurious Tuples

Design relations so that JOIN operations on primary key/foreign key pairs produce correct results. Joining on non-key attributes generates spurious (invalid) tuples.

Note: Example: Use foreign keys to link related tables. Never join on non-key columns.

Q13. Anomalies — INSERT, DELETE, UPDATE Anomalies

Anomalies are problems that arise in a poorly designed database due to data redundancy. Consider a poorly designed relation EMP_DEPT(EmpID, EmpName, Salary, DeptID, DeptName, DeptManager):

EmpID	EmpName	Salary	DeptID	DeptName	DeptManager
E1	Alice	60000	D1	IT	Bob
E2	Charlie	50000	D1	IT	Bob
E3	Diana	70000	D2	HR	Eve

13.1 Insertion Anomaly

Problem: Cannot insert information about a new department (e.g., Finance) unless at least one employee exists in it, because EmpID is the primary key and cannot be NULL.

Also: Inserting a new employee in D1 requires repeating DeptName='IT' and DeptManager='Bob', risking inconsistency.

13.2 Deletion Anomaly

Problem: If Diana (E3) is the only employee in the HR department and we delete her record, all information about the HR department (DeptID=D2, DeptName='HR', DeptManager='Eve') is PERMANENTLY LOST from the database.

13.3 Update (Modification) Anomaly

Problem: If the manager of the IT department changes from Bob to Frank, we must update EVERY row where DeptID='D1' (rows of E1 and E2). If only one row is updated, the database becomes INCONSISTENT — different rows show different managers for the same department.

Solution:

```
-- Decompose into two tables (3NF):  
EMPLOYEE(EmpID, EmpName, Salary, DeptID) -- PK: EmpID; FK: DeptID  
DEPARTMENT(DeptID, DeptName, DeptManager) -- PK: DeptID
```

Q14. How to Avoid Anomalies?

Anomalies arise from poor schema design. The primary way to avoid them is through Normalization — decomposing relations based on functional dependencies.

◆ Apply Normalization (1NF → 2NF → 3NF → BCNF)

Remove partial dependencies (→ 2NF), transitive dependencies (→ 3NF), and remaining superkey violations (→ BCNF). Each normal form eliminates a specific type of anomaly.

◆ Maintain Referential Integrity

Use foreign keys with ON DELETE CASCADE / ON UPDATE CASCADE to keep related tables consistent automatically.

◆ Use Separate Tables for Separate Entities

Never store attributes of two different entities in one table. Follow Guideline 1: one table = one entity.

◆ Use Constraints

Use NOT NULL, UNIQUE, CHECK, PRIMARY KEY, and FOREIGN KEY constraints to enforce data integrity at the schema level.

◆ Use Triggers

For complex business rules that cannot be expressed in constraints, use triggers to automatically enforce them on INSERT, UPDATE, DELETE events.

◆ Design Using ER Modeling

Follow the ER-to-Relational mapping algorithm to systematically produce normalized schemas from a well-designed ER diagram.

Q15. What is Lossless Join? Explain with Example

A decomposition of a relation R into relations R1 and R2 is said to be a LOSSLESS JOIN (or non-additive join) decomposition if the natural join of R1 and R2 always gives back exactly the original relation R — with NO extra (spurious) tuples and NO missing tuples.

Formal Definition:

Decomposition of R into R1 and R2 is lossless if and only if:

Either: $(R1 \cap R2) \rightarrow (R1 - R2)$ [common attributes determine R1's unique attrs]
Or: $(R1 \cap R2) \rightarrow (R2 - R1)$ [common attributes determine R2's unique attrs]

Example of LOSSLESS decomposition:

Original relation R(EmpID, DeptID, DeptName) with FD: DeptID $\rightarrow$ DeptName

R1 = (EmpID, DeptID) R2 = (DeptID, DeptName)

$R1 \cap R2 = \{DeptID\}$

DeptID $\rightarrow$ DeptName (DeptID determines R2 - R1 = {DeptName})

=> Condition satisfied => LOSSLESS JOIN ✓

R1:	EmpID	DeptID	R2:	DeptID	DeptName
	E1	D1		D1	IT
	E2	D1		D2	HR
	E3	D2			

R1 JOIN R2 on DeptID:

EmpID	DeptID	DeptName
E1	D1	IT
E2	D1	IT
E3	D2	HR

=> Exactly = original R ✓

Example of LOSSY decomposition (spurious tuples):

EMP_PROJ(SSN, PNO, HOURS, ENAME, PLOCATION)

Split into: R1(SSN, ENAME, PLOCATION) R2(SSN, PNO, HOURS, PLOCATION)

Common: PLOCATION – but PLOCATION is NOT a key in either table!

=> Joining on PLOCATION generates spurious tuples => LOSSY decomposition ✗

Q16. Checking Equivalence of Two Sets of Functional Dependencies

Two sets of FDs F and G are EQUIVALENT ($F^+ = G^+$) if every FD in F can be derived from G AND every FD in G can be derived from F . We check this using attribute closure (X^+).

Algorithm:

- For each FD $X \rightarrow Y$ in G , compute X^+ under F . If $Y \subseteq X^+$, then F covers G .
- For each FD $A \rightarrow B$ in F , compute A^+ under G . If $B \subseteq A^+$, then G covers F .
- If both F covers G AND G covers F , then $F \equiv G$ (equivalent).

Example — Are F and G equivalent?

$F = \{ A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H \}$

$G = \{ A \rightarrow CD, E \rightarrow AH \}$

Step 1: Check F covers G (every FD in G derivable from F ?)

$A \rightarrow CD$: Compute A^+ under F

Start: $\{A\}$

$A \rightarrow C$: $\{A, C\}$

$AC \rightarrow D$: $\{A, C, D\} \Rightarrow$ contains CD ✓ (F covers $A \rightarrow CD$)

$E \rightarrow AH$: Compute E^+ under F

Start: $\{E\}$

$E \rightarrow AD$: $\{E, A, D\}$

$A \rightarrow C$: $\{E, A, D, C\}$

$AC \rightarrow D$: already have D

$E \rightarrow H$: $\{E, A, D, C, H\} \Rightarrow$ contains A and H ✓ (F covers $E \rightarrow AH$)

$\Rightarrow F$ covers G ✓

Step 2: Check G covers F (every FD in F derivable from G ?)

$A \rightarrow C$: A^+ under G : $A \rightarrow CD \Rightarrow \{A, C, D\} \Rightarrow$ contains C ✓

$AC \rightarrow D$: AC^+ under G : $A \rightarrow CD \Rightarrow \{A, C, D\} \Rightarrow$ contains D ✓

$E \rightarrow AD$: E^+ under G : $E \rightarrow AH \Rightarrow \{E, A, H\}$, $A \rightarrow CD \Rightarrow \{E, A, H, C, D\} \Rightarrow$ has A, D ✓

$E \rightarrow H$: $E^+ = \{E, A, H, C, D\} \Rightarrow$ contains H ✓

$\Rightarrow G$ covers F ✓

CONCLUSION: F and G are EQUIVALENT ($F^+ = G^+$) ✓

Example 2 — Non-equivalent sets:

$F = \{ A \rightarrow B, B \rightarrow C \}$

$G = \{ A \rightarrow B, B \rightarrow C, A \rightarrow C \}$

$A \rightarrow C$ under F : $A^+ = \{A\} \rightarrow B: \{A, B\} \rightarrow C: \{A, B, C\} \Rightarrow$ has C ✓ (F covers $A \rightarrow C$)

All of F is in G trivially $\Rightarrow G$ covers F ✓

$\Rightarrow F$ and G are EQUIVALENT ($A \rightarrow C$ in G is redundant)

Q17. Inference Rules — Armstrong's Axioms with Mathematical Proof

Armstrong's Axioms are a set of inference rules used to derive all valid functional dependencies from a given set. They are both sound (only derive valid FDs) and complete (can derive ALL valid FDs).

IR1 — Reflexivity Rule

Rule: If $Y \subseteq X$, then $X \rightarrow Y$

Proof: If $Y \subseteq X$, then for any two tuples $t1, t2$ with $t1[X]=t2[X]$, we have $t1[Y]=t2[Y]$ since Y is part of X . Hence $X \rightarrow Y$.

Example: Example: $\{A,B,C\} \rightarrow \{A,C\}$ (since $\{A,C\} \subseteq \{A,B,C\}$)

IR2 — Augmentation Rule

Rule: If $X \rightarrow Y$, then $XZ \rightarrow YZ$

Proof: Suppose $X \rightarrow Y$. For tuples $t1, t2$: if $t1[XZ]=t2[XZ]$, then $t1[X]=t2[X]$ (so $t1[Y]=t2[Y]$) and $t1[Z]=t2[Z]$. Therefore $t1[YZ]=t2[YZ]$, so $XZ \rightarrow YZ$.

Example: Example: $A \rightarrow B$ then $AC \rightarrow BC$

IR3 — Transitivity Rule

Rule: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

Proof: Suppose $X \rightarrow Y$ and $Y \rightarrow Z$. For tuples $t1, t2$: if $t1[X]=t2[X]$, then $t1[Y]=t2[Y]$ (by $X \rightarrow Y$), then $t1[Z]=t2[Z]$ (by $Y \rightarrow Z$). Hence $X \rightarrow Z$.

Example: Example: $A \rightarrow B, B \rightarrow C \blacksquare A \rightarrow C$

IR4 — Decomposition Rule

Rule: If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$

Proof: $X \rightarrow YZ$ means X determines both Y and Z . Since $Y \subseteq YZ$, by Reflexivity $YZ \rightarrow Y$, then by Transitivity $X \rightarrow Y$. Similarly $X \rightarrow Z$.

Example: Example: $A \rightarrow BC \blacksquare A \rightarrow B$ and $A \rightarrow C$

IR5 — Union Rule

Rule: If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$

Proof: $X \rightarrow Y$ means for all $t1, t2$ with $t1[X]=t2[X]$, $t1[Y]=t2[Y]$. Similarly $t1[Z]=t2[Z]$. Thus $t1[YZ]=t2[YZ]$, so $X \rightarrow YZ$.

Example: Example: $A \rightarrow B$ and $A \rightarrow C \blacksquare A \rightarrow BC$

IR6 — Pseudotransitivity Rule

Rule: If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$

Proof: From $X \rightarrow Y$, by Augmentation $WX \rightarrow WY$. From $WY \rightarrow Z$, by Transitivity $WX \rightarrow Z$.

Example: Example: $A \rightarrow B, CB \rightarrow D \blacksquare CA \rightarrow D$

Q18. Finding Canonical / Minimal Cover of a Set of FDs

A Minimal Cover (Canonical Cover) F_c of a set of FDs F is a simplified equivalent set where: (1) every FD has a single attribute on the right-hand side, (2) no left-hand side attribute is redundant, (3) no entire FD is redundant.

Algorithm — 3 Steps:

- Step 1 (Right-hand side decomposition): Split every FD $X \rightarrow \{A, B, \dots\}$ into $X \rightarrow A$, $X \rightarrow B$, ... (single RHS attributes)
- Step 2 (Remove redundant LHS attributes): For each FD $X \rightarrow A$ and each B in X , check if $(X - \{B\})^+$ contains A . If yes, B is redundant — remove it.
- Step 3 (Remove redundant FDs): For each FD $X \rightarrow A$, remove it from F temporarily. If A is still in X^+ (using remaining FDs), the FD is redundant — keep it removed.

Example 1:

$F = \{ A \rightarrow BC, B \rightarrow C, AB \rightarrow C, AB \rightarrow D \}$

Step 1 (Split RHS):

$F = \{ A \rightarrow B, A \rightarrow C, B \rightarrow C, AB \rightarrow C, AB \rightarrow D \}$

Step 2 (Remove redundant LHS in $AB \rightarrow C$ and $AB \rightarrow D$):

For $AB \rightarrow C$: Is $A \rightarrow C$ still derivable without B ?

A^+ (without B in LHS) = $A \rightarrow B \rightarrow C \Rightarrow \{A, B, C\}$ contains $C \Rightarrow A$ redundant? No, B is redundant!

Actually test B : $(AB - \{B\})^+ = A^+ = \{A, B, C\}$ contains $C \Rightarrow B$ is redundant in $AB \rightarrow C$

Replace $AB \rightarrow C$ with $A \rightarrow C$ (but $A \rightarrow C$ already exists, so drop $AB \rightarrow C$)

For $AB \rightarrow D$: test B : $A^+ = \{A, B, C\}$ but does not contain $D \Rightarrow B$ is NOT redundant

test A : $B^+ = \{B, C\}$ does not contain $D \Rightarrow A$ is NOT redundant

$AB \rightarrow D$ stays.

Step 3 (Remove redundant FDs):

$A \rightarrow C$: Remove it. $A^+ = \{A, B\}$ (using $A \rightarrow B$ and $B \rightarrow C$) $\Rightarrow$ still get C via transitivity

Wait: $A \rightarrow B, B \rightarrow C \Rightarrow A^+ = \{A, B, C\} \Rightarrow C$ reachable $\Rightarrow A \rightarrow C$ is REDUNDANT, remove it.

Check each remaining FD similarly.

Minimal Cover $F_c = \{ A \rightarrow B, B \rightarrow C, AB \rightarrow D \}$

Example 2 (Exam-style):

$R = \{A, B, C, D, E, F, G\}$

$F = \{ A \rightarrow BC, BC \rightarrow F, BD \rightarrow CE, CD \rightarrow EF, E \rightarrow FG \}$

Step 1: Split RHS

$\{ A \rightarrow B, A \rightarrow C, BC \rightarrow F, BD \rightarrow C, BD \rightarrow E, CD \rightarrow E, CD \rightarrow F, E \rightarrow F, E \rightarrow G \}$

Step 2: Remove redundant LHS attributes

$BC \rightarrow F$: test B alone: $B^+ = \{B\}$ (no FD with B alone) $\Rightarrow C$ not redundant

test C alone: $C^+ = \{C\} \Rightarrow B$ not redundant. $BC \rightarrow F$ stays.

$BD \rightarrow C$: test B : $B^+ = \{B\}$ no $C \Rightarrow$ not redundant

test D : $D^+ = \{D\}$ no $C \Rightarrow$ not redundant. $BD \rightarrow C$ stays.

$BD \rightarrow E$: $B^+ = \{B\}, D^+ = \{D\} \Rightarrow$ no E reachable $\Rightarrow BD \rightarrow E$ stays.

$CD \rightarrow E$: $C^+ = \{C\}, D^+ = \{D\} \Rightarrow CD \rightarrow E$ stays.

$CD \rightarrow F$: $C^+ = \{C\}, D^+ = \{D\} \Rightarrow CD \rightarrow F$ stays.

Step 3: Remove redundant FDs

$A \rightarrow C$: Remove. $A^+ = \{A, B\}$ using $A \rightarrow B$. BC not reachable $\Rightarrow C$ not in $A^+ \Rightarrow$ NOT redundant

Actually: $A \rightarrow B$. Then need C . No way to get C from B alone. Keep $A \rightarrow C$.

$CD \rightarrow F$: Remove. $CD^+ = \{C, D, E, F, G\}$ ($CD \rightarrow E$, $E \rightarrow F$, $E \rightarrow G$) $\Rightarrow F$ reachable $\Rightarrow CD \rightarrow F$ is REDUNDANT

Minimal Cover $F_c = \{ A \rightarrow B, A \rightarrow C, BC \rightarrow F, BD \rightarrow C, BD \rightarrow E, CD \rightarrow E, E \rightarrow F, E \rightarrow G \}$

Note: Always verify: the minimal cover must be equivalent to the original F (same closure F^+).